

Roooter

An OSPF Self-Healing Network Simulator

Prepared By:

Arjun Rathore

2024A4PS0691H

Institution:

Birla Institute of Technology and Science, Hyderabad Campus

Hyderabad, Telangana, India

Date: July 3, 2026

Contents

1	Introduction	2
2	Initial Thinking and Process	2
3	System Architecture and Execution	2
3.1	Phase 1: Algorithmic Mastery & The C++ Core	2
3.2	Phase 2: Systems-Level IPC and Binary Serialization	3
3.3	Phase 3: The Telemetry Agent (Python/FastAPI)	3
3.4	Phase 4: High-Performance Frontend (Vanilla JS)	4
4	Errors Faced and Resolutions	4
4.1	The Windows text mode Corruption Bug	4
4.2	Security AppLocker / Windows Defender Blocking	4
4.3	Chart.js Y-Axis Clipping	5
5	Future Expansion: Multi-Metric QoS	5
6	How to Run the Project	5
7	Conclusion	5

1 Introduction

Modern enterprise networking relies on robust, fault-tolerant systems capable of adapting to hardware failures and traffic congestion in milliseconds. Protocols such as Open Shortest Path First (OSPF) form the backbone of these architectures. The goal of this project, titled **Roooter**, was to elevate a standard algorithmic graph-theory exercise into a production-grade systems engineering application.

This project simulates a high-performance network routing daemon that automatically recalculates optimal data paths in real-time when physical cables fail. By implementing a multi-threaded C++ core, byte-level Inter-Process Communication (IPC) for data serialization, and a modern decoupled telemetry dashboard, the project closely mirrors the architecture of physical routers and enterprise observability platforms.

2 Initial Thinking and Process

The inception of this project began with a critical analysis of standard competitive programming solutions. Typically, shortest-path algorithms (like Dijkstra's) are executed in a single-threaded environment where the graph is static, and the result is printed to the standard output in plaintext.

To make the system self-healing network simulator several architectural leaps were planned:

- **Dynamic Link Failures:** Real networks are not static. The system needed a way to simulate chaos (cables breaking) without stopping the routing engine.
- **Data Serialization:** Passing plain text strings between microservices is inefficient and error-prone. The project required a strict binary protocol to transmit routing tables as raw memory blocks.
- **Observability:** A visual representation was required that separated the heavy lifting (C++) from the metric derivation (Python) and the rendering (JavaScript).

3 System Architecture and Execution

The project was executed in distinct phases, separating the concerns of the algorithmic engine, the data bridge, and the user interface.

3.1 Phase 1: Algorithmic Mastery & The C++ Core

The foundation of Roooter is a highly optimized Dijkstra's algorithm running in C++. It utilizes a Min-Heap (`std::priority_queue`) to achieve a time complexity of $O((V + E) \log V)$.

To introduce network dynamism, the C++ executable was refactored into a multi-threaded daemon.

- **The Chaos Simulator:** A background thread running in an infinite loop continuously selects a random active network edge and sets its weight to infinity (simulating a cut cable). It then triggers a `std::condition_variable` to wake the main thread.
- **Data Safety:** To prevent data races between the simulator breaking a cable and Dijkstra reading the graph, `std::mutex` locks were heavily utilized to guard the network state.

```

1 void networkSimulator(Network &net) {
2     mt19937 rng(time(0));
3     uniform_int_distribution<int> timehealthy(100, 500);
4     uniform_int_distribution<int> timebroken(5000, 10000);
5
6     while (true) {
7         this_thread::sleep_for(chrono::milliseconds(timehealthy(rng)));
8
9         pair<int, int> edgeToBreak = net.getRandomEdge(rng);
10        if (edgeToBreak.first == -1) continue;
11
12        int u = edgeToBreak.first;
13        int v = edgeToBreak.second;
14
15        // Simulate a broken link by setting cost to infinity
16        int originalCost = net.updateEdge(u, v, LS_INFINITY);
17        {
18            // Lock the mutex before notifying the main routing thread
19            lock_guard<mutex> lock(queueMutex);
20            networkChanged = true;
21        }
22        networkEventTrigger.notify_one();
23
24        // Wait for chaotic period to end, then heal the link
25        this_thread::sleep_for(chrono::milliseconds(timebroken(rng)));
26
27        net.updateEdge(u, v, originalCost);
28        {
29            lock_guard<mutex> lock(queueMutex);
30            networkChanged = true;
31        }
32        networkEventTrigger.notify_one();
33    }
34 }

```

Listing 1: Chaos Simulator Thread & Mutex Lock Logic

3.2 Phase 2: Systems-Level IPC and Binary Serialization

Hardware routers do not send text; they send raw binary frames. To mimic this, the standard `std::cout` text streams were replaced with a raw byte-level protocol. Every time the C++ engine finds a path, it constructs a packet with:

1. A 32-bit Sync Word (0xAA55AA55) serving as a packet start beacon.
2. A 32-bit CRC32 checksum, generating a mathematical fingerprint of the payload to detect memory corruption.
3. The payload (Total Cost, Hop Count, and an array of Router IDs).

The C++ code uses `reinterpret_cast<const char*>` to force the compiler to bypass formatting and stream the memory blocks directly into the standard output pipe.

3.3 Phase 3: The Telemetry Agent (Python/FastAPI)

A Python FastAPI server was built to act as the Telemetry Agent and Orchestrator. Using the `subprocess` module, Python boots the C++ executable in the background.

Instead of reading text, Python employs a 1-byte Sliding Window to hunt for the 0xAA55AA55 hex code, perfectly aligning the incoming memory stream. It then uses the `struct.unpack` module to convert the bytes back into integers. Finally, Python derives operational metrics (e.g., Throughput in Mbps and Convergence Time) and streams this data asynchronously via WebSockets.

3.4 Phase 4: High-Performance Frontend (Vanilla JS)

The UI was built without heavy frameworks like React to ensure a 0% CPU footprint and 60 FPS rendering.

- **Vis-Network:** Uses the Barnes-Hut physics algorithm. Nodes untangle like repelling magnets and springs. A lifecycle hook locks the physics engine once stabilized.
- **Chart.js:** Runs on a decoupled `setInterval` heartbeat, creating a smooth-scrolling timeline that accurately drops when throughput vanishes.

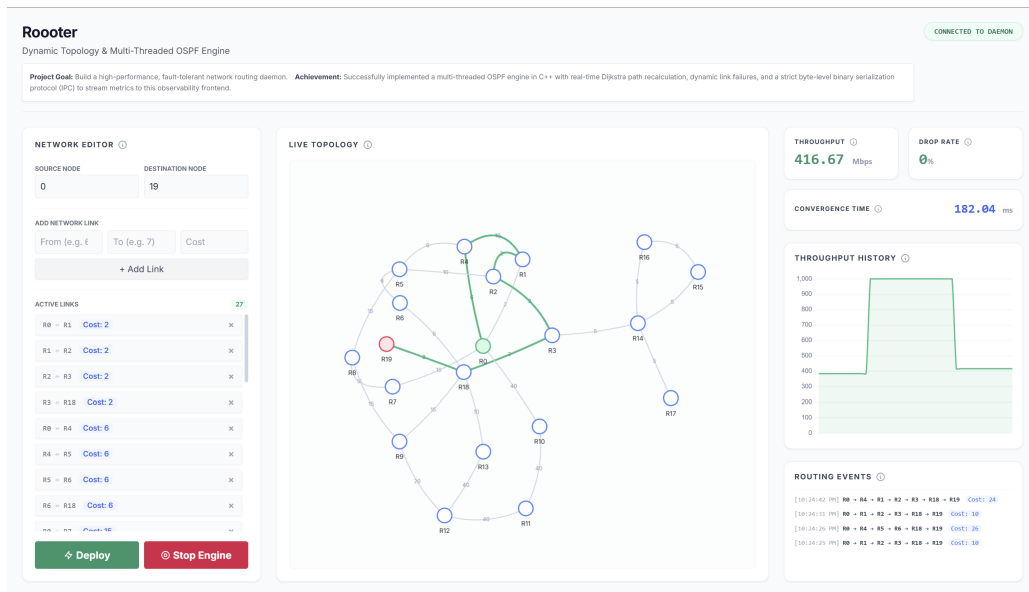


Figure 1: Roooter Frontend Dashboard displaying real-time convergence and throughput.

4 Errors Faced and Resolutions

Developing a cross-language, binary-streaming system introduced several complex bugs:

4.1 The Windows text mode Corruption Bug

Issue: Python was silently dropping all packets. The CRC32 checksums were failing constantly.

Resolution: On Windows, `std::cout` defaults to text mode. If a raw binary byte happens to be 0x0A (newline), Windows secretly injects an extra 0x0D (\r) byte. This corrupted the exact 24-byte alignment. The issue was resolved by injecting `_setmode(_fileno(stdout), _O_BINARY);` at the start of the C++ program to enforce raw output.

4.2 Security AppLocker / Windows Defender Blocking

Issue: Python threw an `OSError: [WinError 4551]` when trying to launch the C++ backend.

Resolution: Because the C++ executable (`a.exe`) was freshly compiled and initiating rapid IPC,

Windows security quarantined it. The fix involved manually unblocking the executable in the File Explorer properties and prepending . to the subprocess path in Python.

4.3 Chart.js Y-Axis Clipping

Issue: When fiber-optic links ($\text{cost} = 2$) were utilized, the throughput spiked to 1000 Mbps, but the chart appeared as a flat line at the top.

Resolution: The Chart.js configuration had a hardcoded `max: 500` limit. This was replaced with `suggestedMax: 700` to allow the Y-axis to scale dynamically with high-bandwidth network routes.

5 Future Expansion: Multi-Metric QoS

While the current engine relies on a single metric (Cost) utilizing standard Dijkstra, real-world SD-WAN infrastructure relies on Quality of Service (QoS).

If this architecture were to be scaled further, the C++ daemon would be upgraded to accept a `traffic_type` flag from the Python UI.

- **VoIP Traffic:** For live, real-time traffic (like a Zoom call), the C++ engine would utilize a Min-Heap to find the path with the absolute lowest *latency*, because any delay causes audio stutter.
- **Bulk Data (The Spotify Use Case):** If a user clicks "Download Playlist" on Spotify to save 10GB of audio for an offline flight, latency becomes irrelevant. The C++ algorithm would dynamically flip to a **Max-Heap** (the "Widest Path" algorithm) to locate the maximum bottleneck *bandwidth* to push massive files through, ignoring standard latency costs.

This dynamic algorithm swapping represents the ultimate frontier of this network simulator.

6 How to Run the Project

To replicate this environment locally, the following environment and commands are required:

1. Compiling the C++ Engine:

The code must be compiled with the Level 3 optimization flag to enable function inlining and loop unrolling.

```
1 g++ -O3 networkdefining_phase3.cpp -o a.exe
```

2. Running the Python Server:

Start the Uvicorn ASGI server to host the FastAPI application and background threads.

```
1 python -m uvicorn api_dashboard:app --reload
```

3. Launching the UI:

Open `index.html` in any modern web browser. Configure the nodes in the Network Editor and click Deploy.

7 Conclusion

The Roooter project successfully bridges the gap between theoretical graph algorithms and practical systems engineering. By implementing binary serialization, multithreading, and a real-time observability frontend, it provides a comprehensive emulation of modern fault-tolerant network architectures.